Inside the Transformer

A Step-by-Step Walkthrough of the Decoder-Only Architecture

CS 115B – Spring 2026

James Pustejovsky

March 31, 2026

What This Lecture Is

We have talked *about* Transformers.

Today we walk *through* one.

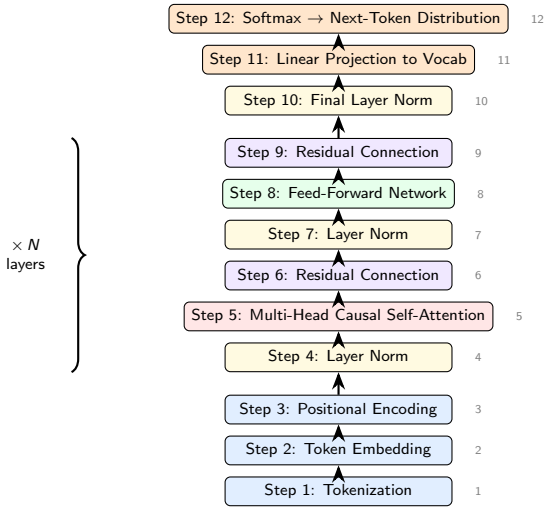
Goal: Trace the journey of a single input sequence from raw text to predicted next token, component by component, in the exact order that computation happens.

Architecture: Decoder-only (GPT-style).

Plan:

1. Original 2017 components (Vaswani et al.)
2. Then: what modern models (LLaMA, etc.) changed and why

The Full Picture (We'll Unpack Every Box)



Step 1: Tokenization

Raw text in, integer IDs out.

Example (BPE-style):

```
"The cat sat on the mat"
                ↓
["The", " cat", " sat", " on", " the", " mat"]
                ↓
[464, 3797, 3332, 319, 262, 2603]
```

Key points:

- Subword tokenization (BPE, SentencePiece, tiktoken) – not word-level, not character-level
- Vocabulary size V is fixed (e.g., GPT-2: 50,257; LLaMA: 32,000)
- Every input to the model is a sequence of integers: $[t_1, t_2, \ldots, t_n]$
- The model never sees raw characters – only token IDs

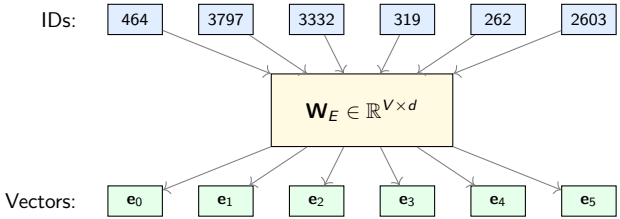
Output: $\mathbf{t} \in \{0, 1, \ldots, V-1\}^n$

Step 2: Token Embedding

Integer IDs in, dense vectors out.

A learned lookup table $\mathbf{W}_E \in \mathbb{R}^{V \times d}$:

$$\mathbf{e}_i = \mathbf{W}_E[t_i, :] \qquad \in \mathbb{R}^d$$



Each token ID is replaced by a d-dimensional vector (e.g., $d = 768$ for GPT-2 small).

Output: $\mathbf{E} = [\mathbf{e}_1; \mathbf{e}_2; \ldots; \mathbf{e}_n] \in \mathbb{R}^{n \times d}$

Step 3: Positional Encoding (Original)

Problem: Self-attention is permutation-invariant – it treats the input as a *set*.

Solution: Add a position signal to each embedding.

Sinusoidal positional encoding (Vaswani et al., 2017):

$$\mathrm{PE}(p, 2i) = \sin\left(\frac{p}{10000^{2i/d}}\right)$$

$$\mathrm{PE}(p, 2i+1) = \cos\left(\frac{p}{10000^{2i/d}}\right)$$

where p = position index, i = dimension index.

Add to embeddings (element-wise):

$$\mathbf{X}^{(0)} = \mathbf{E} + \mathbf{PE} \qquad \in \mathbb{R}^{n \times d}$$

This is the input to the first Transformer layer.
$\mathbf{X}^{(0)}$ now encodes both *what* each token is and *where* it is.

Why Sinusoids?

Each dimension oscillates at a different frequency:

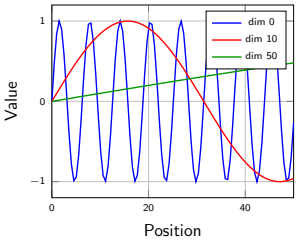
- Low dimensions: long wavelengths (slowly varying)
- High dimensions: short wavelengths (rapidly varying)

Key property: For any fixed offset k, there exists a linear transformation $\mathbf{M}_k$ such that:

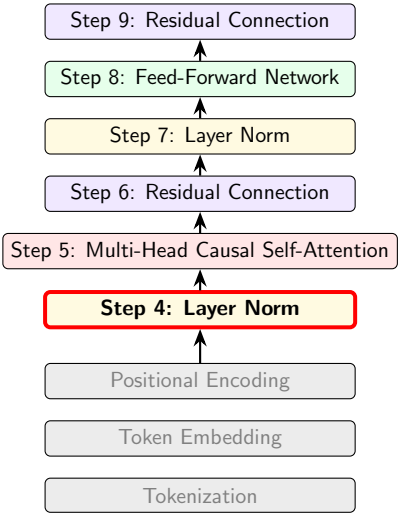
$$\text{PE}(p+k) = \mathbf{M}_k \cdot \text{PE}(p)$$

The model can learn to attend to relative positions via linear operations on these encodings.

GPT-2 in practice: Used *learned* positional embeddings instead ($\mathbf{W}_P \in \mathbb{R}^{n_{\max} \times d}$), but the principle is the same.



Entering the Layer



⟵ **You are here**

Input to the layer: $\mathbf{X}^{(\ell-1)} \in \mathbb{R}^{n \times d}$ (output of the previous layer, or $\mathbf{X}^{(0)}$ for the first layer).

Step 4: Layer Normalization

Normalize each token's representation independently.

For each token vector $\mathbf{x} \in \mathbb{R}^d$:

$$\text{LayerNorm}(\mathbf{x}) = \boldsymbol{\gamma} \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \boldsymbol{\beta}$$

where:

- $\mu = \frac{1}{d}\sum_{j=1}^{d} x_j$ (mean across d dimensions)
- $\sigma^2 = \frac{1}{d}\sum_{j=1}^{d}(x_j - \mu)^2$ (variance across d dimensions)
- $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$: learned scale and shift; $\epsilon \approx 10^{-5}$

Why?

- Stabilizes activations – prevents values from drifting to extreme ranges
- Each token normalized *independently* (unlike BatchNorm across the batch)
- Applied *before* each sub-layer (pre-norm) in GPT-2 and all modern models

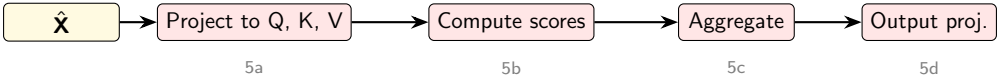
Output: $\hat{\mathbf{X}} = \text{LayerNorm}(\mathbf{X}^{(\ell-1)})$

Step 5: Multi-Head Self-Attention – Overview

This is the core mechanism. Let's go through it carefully.

The attention sub-layer does three things:

1. **Project** the normalized input into Queries, Keys, and Values
2. **Compute** attention scores (with causal mask)
3. **Aggregate** values weighted by attention, across multiple heads



Step 5a: Project to Q, K, V

Three separate learned linear projections, applied to every token:

$$\mathbf{Q} = \hat{\mathbf{X}}\mathbf{W}_Q, \qquad \mathbf{K} = \hat{\mathbf{X}}\mathbf{W}_K, \qquad \mathbf{V} = \hat{\mathbf{X}}\mathbf{W}_V$$

where $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d \times d}$.

Intuition:

- **Query** ($\mathbf{q}_i$): "What am I looking for?"
- **Key** ($\mathbf{k}_j$): "What do I contain?"
- **Value** ($\mathbf{v}_j$): "What information do I contribute if attended to?"

Multi-head: Split $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ into h heads.
Each head operates on dimension $d_k = d/h$:

$$\mathbf{Q}^{(i)} \in \mathbb{R}^{n \times d_k}, \quad \mathbf{K}^{(i)} \in \mathbb{R}^{n \times d_k}, \quad \mathbf{V}^{(i)} \in \mathbb{R}^{n \times d_k}$$

GPT-2 small: $d = 768$, $h = 12$, so $d_k = 64$.

Step 5b: Scaled Dot-Product Attention (One Head)

For a single attention head i:

$$\text{Attention}(\mathbf{Q}^{(i)}, \mathbf{K}^{(i)}, \mathbf{V}^{(i)}) = \text{softmax}\left(\frac{\mathbf{Q}^{(i)}{\mathbf{K}^{(i)}}^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}^{(i)}$$

Step by step:

1. $\mathbf{S} = \mathbf{Q}^{(i)}{\mathbf{K}^{(i)}}^\top \in \mathbb{R}^{n \times n}$ (raw similarity scores)
2. $\mathbf{S} \leftarrow \mathbf{S} / \sqrt{d_k}$ (scale to prevent softmax saturation)
3. $\mathbf{S} \leftarrow \mathbf{S} + \mathbf{M}$ (apply causal mask)
4. $\mathbf{A} = \text{softmax}(\mathbf{S})$ (row-wise softmax $\rightarrow$ attention weights)
5. $\text{head}_i = \mathbf{A}\,\mathbf{V}^{(i)}$ (weighted sum of value vectors)

S_{pq} = "how much should token p attend to token q?"
A_{pq} = the actual weight (after softmax, sums to 1 over q).
$\text{head}_i[p, :] = \sum_q A_{pq} \cdot \mathbf{v}_q^{(i)}$

The Causal Mask

Decoder-only models must not look into the future.

The causal mask $\mathbf{M} \in \mathbb{R}^{n \times n}$: $\quad M_{pq} = \begin{cases} 0 & \text{if } p \geq q \\ -\infty & \text{if } p < q \end{cases}$

Query \ Key	t_1	t_2	t_3	t_4	t_5
t_1	0	$-\infty$	$-\infty$	$-\infty$	$-\infty$
t_2	0	0	$-\infty$	$-\infty$	$-\infty$
t_3	0	0	0	$-\infty$	$-\infty$
t_4	0	0	0	0	$-\infty$
t_5	0	0	0	0	0

After $\mathbf{S} + \mathbf{M}$, softmax turns $-\infty$ entries into 0.

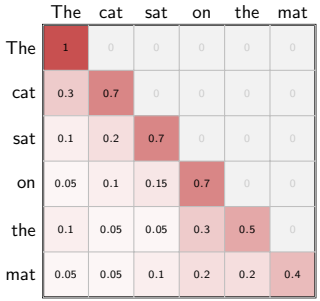
Result: token p attends **only** to tokens $1, \ldots, p$.

Token 1 sees only itself.
Token 3 sees tokens 1, 2, 3.
Token 5 sees all five tokens.

This is what makes the model **autoregressive**.

Visualizing Attention Weights

After softmax, each row of $\mathbf{A}$ is a probability distribution:



Each row sums to 1.

Darker = higher attention weight.

Reading row "mat":
it attends most strongly to itself (0.40) and "on" and "the" (0.20 each).

This is *one head*. Different heads learn different patterns (syntax, coreference, positional, etc.).

Step 5c–5d: Concatenate Heads and Output Projection

5c. Concatenate all h head outputs:

$$\text{MultiHead} = \text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head}_h) \in \mathbb{R}^{n \times d}$$

Each $\text{head}_i \in \mathbb{R}^{n \times d_k}$; concatenation restores the full dimension $d = h \cdot d_k$.

5d. One final linear projection:

$$\mathbf{Z}_{\text{attn}} = \text{MultiHead} \cdot \mathbf{W}_O \qquad \mathbf{W}_O \in \mathbb{R}^{d \times d}$$

Why multiple heads?

- Each head has its own $\mathbf{W}_Q^{(i)}$, $\mathbf{W}_K^{(i)}$, $\mathbf{W}_V^{(i)}$
- Heads learn different attention patterns in parallel
- One head might track syntax; another tracks coreference; another tracks local context
- $\mathbf{W}_O$ mixes head outputs into a unified representation

Output: $\mathbf{Z}_{\text{attn}} \in \mathbb{R}^{n \times d}$ (same shape as input)

Step 6: First Residual Connection

$$\mathbf{X}_{\text{mid}} = \mathbf{X}^{(\ell-1)} + \mathbf{Z}_{\text{attn}}$$



Why residual connections?

- Gradient highway: gradients flow directly through the skip connection, avoiding vanishing gradients across many layers
- Each layer makes an *incremental update* to the representation, rather than overwriting it
- Without residuals, deep Transformers ($N > 6$) do not train
- Mathematically: each layer computes $f(\mathbf{x}) + \mathbf{x}$, so at worst a layer can learn $f = 0$ (identity)

Step 7: Second Layer Norm

$$\hat{\mathbf{X}}_{\text{mid}} = \text{LayerNorm}(\mathbf{X}_{\text{mid}})$$

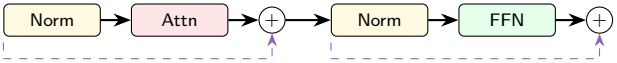
Same operation as Step 4, with its own learned $\boldsymbol{\gamma}$, $\boldsymbol{\beta}$.

Purpose: Re-normalize before the next sub-layer (the FFN).

Pattern: In every Transformer layer, the structure is:

$$\text{Norm} \rightarrow \text{Sub-layer} \rightarrow \text{Add residual}$$

repeated twice (once for attention, once for FFN).



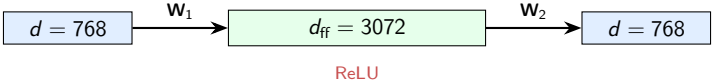
Step 8: The Feed-Forward Network

A two-layer MLP applied independently to each token.

$$\text{FFN}(\mathbf{x}) = \mathbf{W}_2 \, \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

where:

- $\mathbf{W}_1 \in \mathbb{R}^{d \times d_{\text{ff}}}$ (expand: $d \to d_{\text{ff}}$)
- $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d}$ (contract: $d_{\text{ff}} \to d$)
- Typically $d_{\text{ff}} = 4d$ (GPT-2 small: $768 \to 3072 \to 768$)



Key point: The FFN is *position-wise* – identical function applied to each token independently. No token-to-token interaction here. That's the attention layer's job.

Interpretation: FFN stores and retrieves "knowledge" (factual associations).

Attention vs. FFN: Two Different Roles

Self-Attention (Step 5)	**Feed-Forward (Step 8)**
Mixes information *across tokens*	Transforms each token *independently*
$n \times n$ interaction matrix	Applied position-wise (no cross-talk)
"Communication"	"Computation"
Learns *which* tokens to combine	Learns *what* to compute with the combined info
Thought of as "routing"	Thought of as "knowledge storage"
Parameters: $4d^2$ (for Q, K, V, O)	Parameters: $2 \cdot d \cdot d_{\text{ff}} = 8d^2$

Note: The FFN has *twice as many* parameters as the attention layer!

Recent work (Geva et al., 2021; Meng et al., 2022) shows that FFN rows store factual associations: specific key patterns in $\mathbf{W}_1$ activate specific value patterns in $\mathbf{W}_2$.

Step 9: Second Residual Connection

$$\mathbf{X}^{(\ell)} = \mathbf{X}_{\text{mid}} + \text{FFN}(\hat{\mathbf{X}}_{\text{mid}})$$

$\mathbf{X}^{(\ell)}$ is the output of layer ℓ and the input to layer $\ell + 1$.

Putting the full layer together:

$$\begin{aligned}
\hat{\mathbf{X}} &= \text{LayerNorm}(\mathbf{X}^{(\ell-1)}) \\
\mathbf{X}_{\text{mid}} &= \mathbf{X}^{(\ell-1)} + \text{MultiHeadAttn}(\hat{\mathbf{X}}) \\
\hat{\mathbf{X}}_{\text{mid}} &= \text{LayerNorm}(\mathbf{X}_{\text{mid}}) \\
\mathbf{X}^{(\ell)} &= \mathbf{X}_{\text{mid}} + \text{FFN}(\hat{\mathbf{X}}_{\text{mid}})
\end{aligned}$$

That's it. Four equations. One Transformer layer. Repeat N times.

$\times N$: Stacking Layers

The input passes through N identical (but independently parameterized) layers:

$$\mathbf{X}^{(0)} \xrightarrow{\text{Layer 1}} \mathbf{X}^{(1)} \xrightarrow{\text{Layer 2}} \mathbf{X}^{(2)} \xrightarrow{\cdots} \mathbf{X}^{(N)}$$

Model	N	d
GPT-2 Small	12	768
GPT-2 XL	48	1600
GPT-3 (175B)	96	12,288
LLaMA-2 7B	32	4,096
LLaMA-2 70B	80	8,192

What changes across layers?

- Early layers: local syntax, POS-like features
- Middle layers: semantic composition, entity tracking
- Late layers: task-specific, prediction-oriented features

Step 10: Final Layer Norm

$$\hat{\mathbf{X}}^{(N)} = \text{LayerNorm}(\mathbf{X}^{(N)})$$

After the last layer, one more LayerNorm before the output projection.

(With its own learned γ, β.)

Why here? The pre-norm convention means the last sub-layer's residual output has not been normalized. This final norm ensures the representations are in a good range before the linear projection to vocabulary.

Step 11: Linear Projection to Vocabulary

$$\mathbf{L} = \hat{\mathbf{X}}^{(N)}\mathbf{W}_{\text{vocab}} \qquad \in \mathbb{R}^{n \times V}$$

where $\mathbf{W}_{\text{vocab}} \in \mathbb{R}^{d \times V}$.

Each token position gets a **logit** (raw score) for every token in the vocabulary.

Weight tying (common): $\mathbf{W}_{\text{vocab}} = \mathbf{W}_E^\top$ – the same matrix used for input embeddings, transposed.

- Reduces parameters by $d \times V$ (e.g., $768 \times 50{,}257 \approx 38$M for GPT-2)
- Enforces consistency: tokens with similar embeddings get similar logits

$L_{p,v}$ = "how much does position p predict token v as the next token?"

Step 12: Softmax and Next-Token Prediction

$$P(x_{p+1} = v \mid x_1, \ldots, x_p) = \text{softmax}(\mathbf{L}_{p,:})_v = \frac{e^{L_{p,v}}}{\sum_{v'=1}^{V} e^{L_{p,v'}}}$$



During training: Compute cross-entropy loss at *every* position:

$$\mathcal{L} = -\frac{1}{n} \sum_{p=1}^{n} \log P(x_{p+1} \mid x_1, \ldots, x_p)$$

Thanks to the causal mask, all n predictions are computed in a single forward pass.

During generation: Take the logits at the *last* position, sample from the distribution (with temperature, top-k, top-p), append the token, repeat.

The Complete Forward Pass

Input: token IDs $[t_1, \ldots, t_n]$

1. $\mathbf{E} = \text{Embed}([t_1, \ldots, t_n]) \in \mathbb{R}^{n \times d}$
2–3. $\mathbf{X}^{(0)} = \mathbf{E} + \mathbf{PE}$

For $\ell = 1, \ldots, N$:

4. $\hat{\mathbf{X}} = \text{LayerNorm}(\mathbf{X}^{(\ell-1)})$
5. $\mathbf{Z} = \text{MHCausalAttn}(\hat{\mathbf{X}})$
6. $\mathbf{X}_{\text{mid}} = \mathbf{X}^{(\ell-1)} + \mathbf{Z}$
7. $\hat{\mathbf{X}}_{\text{mid}} = \text{LayerNorm}(\mathbf{X}_{\text{mid}})$
8. $\mathbf{F} = \text{FFN}(\hat{\mathbf{X}}_{\text{mid}})$
9. $\mathbf{X}^{(\ell)} = \mathbf{X}_{\text{mid}} + \mathbf{F}$

10. $\hat{\mathbf{X}}^{(N)} = \text{LayerNorm}(\mathbf{X}^{(N)})$
11. $\mathbf{L} = \hat{\mathbf{X}}^{(N)} \mathbf{W}_{\text{vocab}}$
12. $P(x_{p+1} \mid \cdot) = \text{softmax}(\mathbf{L}_{p,:})$

Parameter Count: Where Do the Parameters Live?

GPT-2 Small: $d = 768$, $h = 12$, $d_{\text{ff}} = 3072$, $V = 50{,}257$, $N = 12$.

Component	**Params**	**Formula**
Token embeddings $\mathbf{W}_E$	38.6M	$V \times d$
Position embeddings $\mathbf{W}_P$	0.8M	$n_{\text{max}} \times d$
Per layer:		
Attention (Q, K, V, O)	2.4M	$4d^2$
FFN ($\mathbf{W}_1$, $\mathbf{W}_2$, biases)	4.7M	$2d \cdot d_{\text{ff}}$
LayerNorms ($\times 2$)	3K	$2 \times 2d$
Layer total	**7.1M**	
All $N = 12$ layers	85.1M	
Final LayerNorm + output (tied)	$\sim$0	
Total	**$\sim$124M**	

Observation: $\sim 2/3$ of each layer's parameters are in the FFN, not the attention.

What Changed?

The 12-step pipeline is *the same*.

The individual components got upgraded.

Step	Original (2017–2019)	Modern (2023+)
3. Positional	Sinusoidal / Learned	RoPE
4,7. Normalization	LayerNorm	RMSNorm
5. Attention	Multi-Head (MHA)	Grouped-Query (GQA)
8. FFN activation	ReLU	SwiGLU
Norm placement: post-norm (2017) $\rightarrow$ pre-norm (GPT-2+) $\rightarrow$ unchanged in modern		

Let's go through each upgrade.

Update 1: RMSNorm (Replacing LayerNorm)

Original LayerNorm: Center (subtract mean), scale (divide by std), affine transform.

$$\text{LayerNorm}(\mathbf{x}) = \boldsymbol{\gamma} \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \boldsymbol{\beta}$$

RMSNorm (Zhang & Sennrich, 2019): Drop the mean centering and bias.

$$\text{RMSNorm}(\mathbf{x}) = \boldsymbol{\gamma} \odot \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \qquad \text{where RMS}(\mathbf{x}) = \sqrt{\frac{1}{d}\sum_{j=1}^{d} x_j^2}$$

Why the change?

- Slightly faster (no mean computation, no $\boldsymbol{\beta}$ parameter)
- Empirically equivalent or better performance
- The re-centering in LayerNorm was found to be unnecessary in practice
- Fewer parameters: d instead of $2d$ per norm

Update 2: Rotary Positional Embeddings (RoPE)

Original: Add position vectors to token embeddings *once*, before the first layer.

RoPE (Su et al., 2021): Encode position by *rotating* Q and K vectors *inside every attention layer*.

For position m and dimension pair $(2i, 2i+1)$:

$$\begin{pmatrix} q'_{2i} \\ q'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{pmatrix} \begin{pmatrix} q_{2i} \\ q_{2i+1} \end{pmatrix} \qquad \theta_i = 10000^{-2i/d}$$

Applied to both $\mathbf{q}$ and $\mathbf{k}$ before computing $\mathbf{q}^\top \mathbf{k}$.

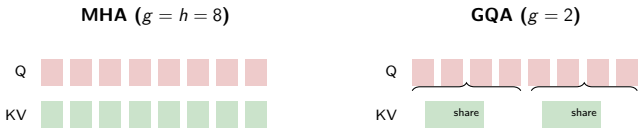
Key property: $\langle R_m \mathbf{q},\ R_n \mathbf{k} \rangle = f(\mathbf{q}, \mathbf{k}, m-n)$
The dot product depends only on *relative* position $m - n$.

Advantages: Naturally relative; extrapolates to longer sequences (with NTK-aware scaling); no added parameters.

Update 3: Grouped-Query Attention (GQA)

Original MHA: h query heads, h key heads, h value heads.

GQA: h query heads, g key/value heads ($g < h$; groups of h/g queries share one KV head).



Why? The KV-cache during inference stores **K** and **V** for all past tokens.
Reducing from h to g KV heads cuts KV-cache memory by h/g.

LLaMA-2 70B: $h = 64$ query heads, $g = 8$ KV heads $\rightarrow 8\times$ KV-cache reduction.
Negligible quality loss.

Update 4: SwiGLU (Replacing ReLU FFN)

Original FFN:

$$\text{FFN}(\mathbf{x}) = \mathbf{W}_2\,\text{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

SwiGLU (Shazeer, 2020):

$$\text{FFN}(\mathbf{x}) = \mathbf{W}_2\Big[\text{SiLU}(\mathbf{W}_{\text{gate}}\,\mathbf{x}) \odot (\mathbf{W}_1\mathbf{x})\Big]$$

where $\text{SiLU}(z) = z \cdot \sigma(z)$ (smooth approximation to ReLU), and $\odot$ is element-wise multiplication.

Three projections instead of two: $\mathbf{W}_1$, $\mathbf{W}_{\text{gate}}$, $\mathbf{W}_2$.

To keep total parameters comparable, d_{ff} is reduced (e.g., $\frac{8}{3}d$ instead of $4d$).

Why it helps:

- Gating mechanism: the network learns to *selectively pass* information
- SiLU is smooth everywhere (no dead neurons like ReLU)
- Consistent improvements across all benchmarks

No biases in modern models (LLaMA, Mistral, etc.) – further simplification

The Modern Layer: Side by Side

Original (GPT-2, 2019)

$$\hat{\mathbf{X}} = \text{LayerNorm}(\mathbf{X}^{(\ell-1)})$$
$$\mathbf{Z} = \text{MHA}(\hat{\mathbf{X}})$$
$$\mathbf{X}_m = \mathbf{X}^{(\ell-1)} + \mathbf{Z}$$
$$\hat{\mathbf{X}}_m = \text{LayerNorm}(\mathbf{X}_m)$$
$$\mathbf{F} = \text{ReLU-FFN}(\hat{\mathbf{X}}_m)$$
$$\mathbf{X}^{(\ell)} = \mathbf{X}_m + \mathbf{F}$$

Sinusoidal/learned positions
added once at input

Modern (LLaMA, 2023+)

$$\hat{\mathbf{X}} = \text{RMSNorm}(\mathbf{X}^{(\ell-1)})$$
$$\mathbf{Z} = \text{GQA+RoPE}(\hat{\mathbf{X}})$$
$$\mathbf{X}_m = \mathbf{X}^{(\ell-1)} + \mathbf{Z}$$
$$\hat{\mathbf{X}}_m = \text{RMSNorm}(\mathbf{X}_m)$$
$$\mathbf{F} = \text{SwiGLU-FFN}(\hat{\mathbf{X}}_m)$$
$$\mathbf{X}^{(\ell)} = \mathbf{X}_m + \mathbf{F}$$

RoPE applied inside attention
at every layer

Same skeleton. Four component upgrades. That's the whole story.

Summary: The 12 Steps

1. **Tokenize**: raw text → integer IDs
2. **Embed**: IDs → dense vectors via lookup
3. **Add position**: sinusoidal / learned / RoPE

— *Repeat N times:*

4. **Layer Norm**: normalize each token vector
5. **Multi-Head Causal Self-Attention**: tokens communicate
6. **Residual**: add skip connection
7. **Layer Norm**: normalize again
8. **FFN**: each token transformed independently
9. **Residual**: add skip connection

— *End loop*

10. **Final Layer Norm**
11. **Linear projection**: $d \to V$ (logits over vocabulary)
12. **Softmax**: logits → probability distribution → next token

Key Takeaways

1. The decoder-only Transformer is remarkably **simple**: embeddings, repeated blocks of attention + FFN with norms and residuals, then a projection.
2. **Attention** handles token-to-token interaction. **FFN** handles per-token transformation. These are the only two operations.
3. The **causal mask** is what makes the architecture autoregressive – it's just one triangular matrix of $-\infty$'s.
4. **Residual connections** are critical – they enable depth and provide gradient highways.
5. Modern upgrades (RMSNorm, RoPE, GQA, SwiGLU) refine the components but **do not change the architecture's skeleton**.
6. Everything in the forward pass is **matrix multiplications and element-wise operations** – that's why GPUs can do it fast.